ASSIGNMENT-1

NAME:- M CHARAN KUMAR REDDY

REG.NO.:192411080

COURSE:-DSA0110- OBJECT ORIENTED PROGRAMMING WITH C++

TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

Object-Oriented Analysis

To model the Namma Metro Smart-Card system robustly, we identify the following core domain objects.

SmartCard (Card):

Data: cardId (string/long), balance (long long, stored in paise), ownerType (Regular/Student/Senior).

Behaviour: deduct(amount), recharge(amount), getBalance(), canTravel(fare).

Journey (Trip):

Data: sourceStation, destinationStation, farthestZone (Zone), timestamp (hour), isPeakHour (bool).

Behaviour: calculateFare(), validateJourney().

FareTable:

Data: Base fares per zone (Inner, Middle, Outer), peak surcharge percentage (20%), minimum balance threshold (Rs 50).

Behaviour: getBaseFare(Zone), getSurcharge(bool).

Passenger:

Data: passengerId, category (General, Student, Senior Citizen).

Behaviour: applyConcession().

Procedural vs. Object-Oriented Style

Procedural Style: Uses global variables (e.g., long long global_balance;) and loose functions (long long compute_fare(...), bool check_balance(...)). Data and functions are decoupled, leading to vulnerable state management where any function can accidentally modify the balance directly, causing bugs like negative balances.

Object-Oriented Style: Encapsulates data and behaviors inside classes (SmartCard). The balance is marked private, preventing external tampering. State transitions happen strictly through verified member functions (deduct()).

Pillars of OOP in the Metro System

Encapsulation: Hiding card balance state inside SmartCard and exposing controlled mutators (deduct).

Abstraction: Hiding complex zone-routing algorithms behind simple method calls like journey.calculateFare().

Inheritance: Base class Passenger inherited by specialized classes StudentPassenger and SeniorCitizenPassenger to handle concessions polymorphically.

Polymorphism: A virtual calculateDiscountedFare() method overridden in subclass types to compute customized fares.

Core Business Rules

Minimum Balance Rule: A passenger's card balance *after* fare deduction must never fall below Rs 50.00 (5000 paise).

Exact Fare Rule: All monetary calculations must use integer paise to avoid IEEE-754 floating-point rounding errors.

Peak Hour Definition: Peak hours are strictly 8 <= hour < 10 and 17 <= hour < 20.

PROGRAM CODE;-

```
#include <iostream>

#include <string>

class SmartCard {

private:

   long long balancePaise;

public:

   explicit SmartCard(long long initialBalance) : balancePaise(initialBalance) {}

   long long getBalance() const { return balancePaise; }

   void deduct(long long amount) { balancePaise -= amount; }

};

int main() {
```

std::cout << "=== Task 1: OOP Analysis & Architecture ===" << std::endl;

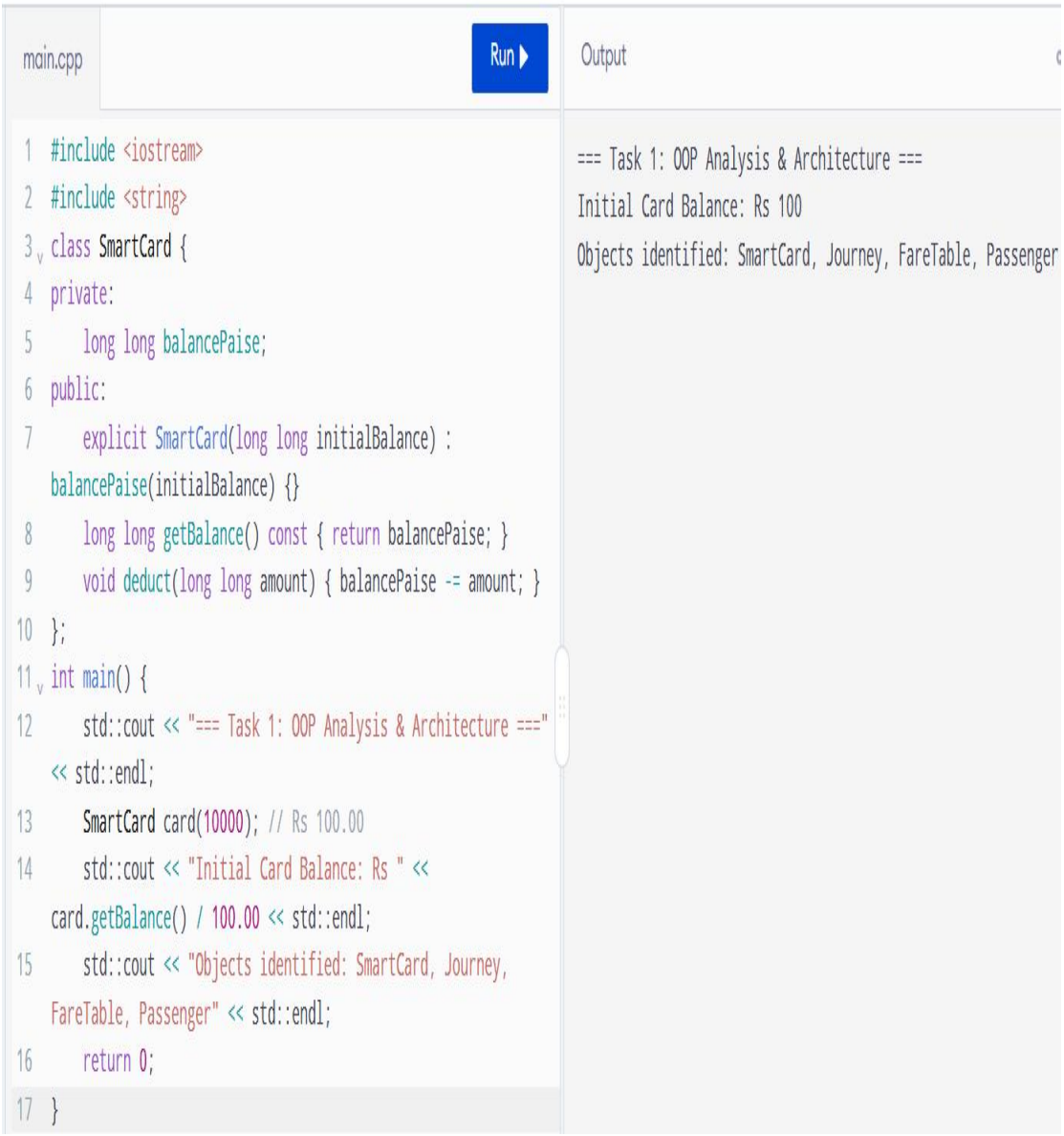
SmartCard card(10000); // Rs 100.00

std::cout << "Initial Card Balance: Rs " << card.getBalance() / 100.00 << std::endl;

std::cout << "Objects identified: SmartCard, Journey, FareTable, Passenger" << std::endl;

return 0;

}



TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

Discussion on Tokens and Constants

Five Kinds of Tokens:

Keywords: constexpr, enum, class, long, return.

Identifiers: BASE_PAISE, Zone, canTravel.

Constants: 1000, 5000, 20.

Operators: +, -, ==, &&.

Punctuators: {, }, ;, ,.

#define vs const vs constexpr: #define is a preprocessor macro lacking type safety and scope. const defines run-time constants. constexpr enforces compile-time evaluation, guaranteeing memory efficiency and zero runtime overhead, making it ideal for static fare tables.

enum vs enum class: Standard enums leak enumerators into the enclosing scope and allow implicit integer conversions. enum class provides strong scoping (Zone::Inner) and strict type safety.

PROGRAM CODE:-

```
// Task 01: Understand the Problem and Identify the Objects

#include <iostream>

#include <string>

class SmartCard {

private:

   long long balancePaise;

public:

   explicit SmartCard(long long initialBalance) : balancePaise(initialBalance) {}

   long long getBalance() const { return balancePaise; }

   void deduct(long long amount) { balancePaise -= amount; }

};
```

int main() {

std::cout << "=== Task 1: OOP Analysis & Architecture ===" << std::endl;

SmartCard card(10000); // Rs 100.00

std::cout << "Initial Card Balance: Rs " << card.getBalance() / 100.00 << std::endl;

std::cout << "Objects identified: SmartCard, Journey, FareTable, Passenger" << std::endl;

return 0;

}

main.cpp

Run

Output

```
#include <iostream>
using namespace std;
#ifndef FARE_CONSTANTS_H
#define FARE_CONSTANTS_H
enum class Zone { Inner = 1, Middle, Outer };
constexpr long long BASE_PAISE[] = { 1000, 2000, 3000 };
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
#endif
int main() {
    std::cout << "--- Fare System Constants ---" << std::endl;
    std::cout << "Base Fare (Inner Zone): " << BASE_PAISE[0] << " paise" << std::endl;
    std::cout << "Peak Surcharge: " << PEAK_SURCHARGE_PERCENT << "%" << std::endl;
    std::cout << "Minimum Balance Required: " << MIN_BALANCE_PAISE << " paise" << std::endl;
    return 0;
}
```

```
--- Fare System Constants ---
Base Fare (Inner Zone): 1000 paise
Peak Surcharge: 20%
Minimum Balance Required: 5000 paise
```

Task 3: Data Types and Paisa Storage Validation

PROGRAM CODE:-

```
#include <iostream>

#include <climits>


int main() {

    std::cout << "=== Task 3: Data Types & Sizes ===" << std::endl;

    std::cout << "sizeof(long long) for Money: " << sizeof(long long) << " bytes" << std::endl;

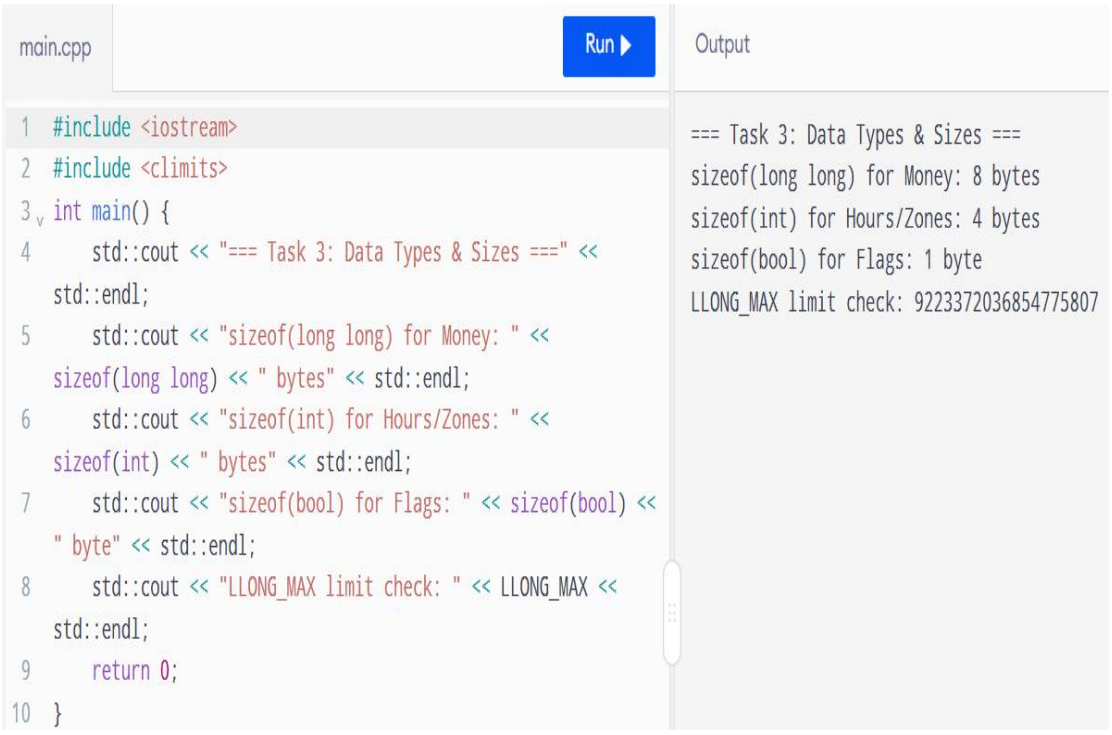
    std::cout << "sizeof(int) for Hours/Zones: " << sizeof(int) << " bytes" << std::endl;

    std::cout << "sizeof(bool) for Flags: " << sizeof(bool) << " byte" << std::endl;

    std::cout << "LLONG_MAX limit check: " << LLONG_MAX << std::endl;

    return 0;

}
```



Task 4: Control Implicit Conversions (static_cast)

PROGRAM CODE:-

```
// Task 04: Control Implicit Conversions

#include <iostream>

int main() {

   std::cout << "=== Task 4: Implicit vs Explicit Conversions ===" << std::endl;

   int a = 7, b = 2;

   double c = a / b; // Integer division truncated

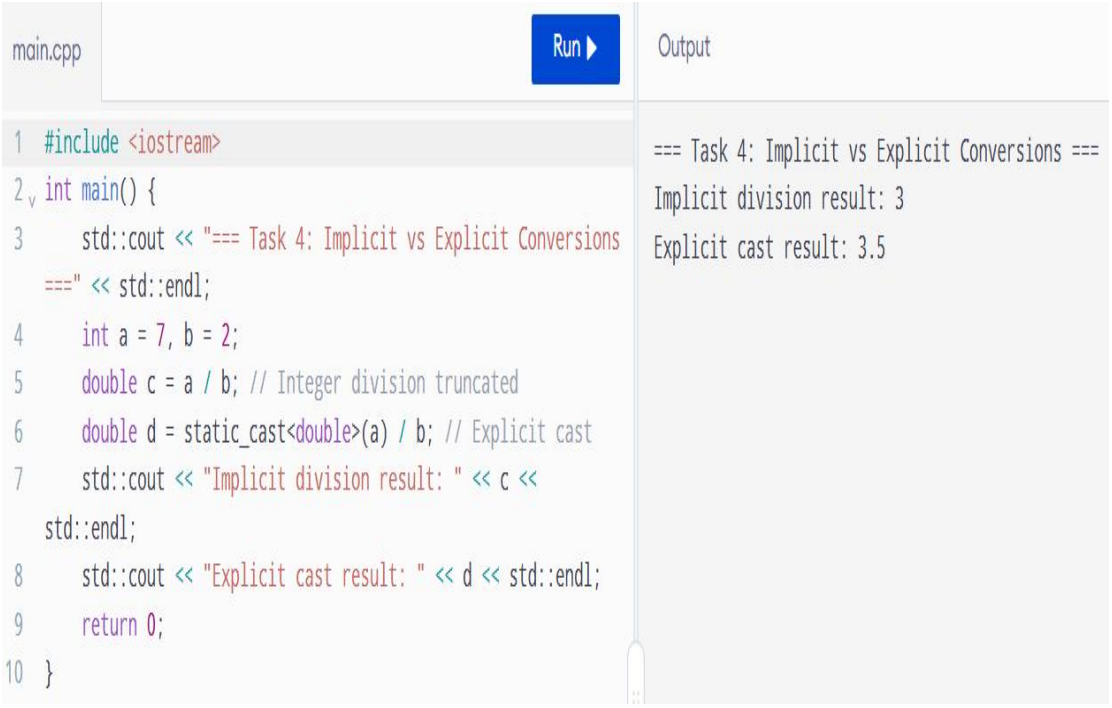
   double d = static_cast<double>(a) / b; // Explicit cast

   std::cout << "Implicit division result: " << c << std::endl;

   std::cout << "Explicit cast result: " << d << std::endl;

   return 0;

}
```



Task 5: Operators, Precedence & Peak Expressions:-

PROGRAM CODE:-

```
#include <iostream>

int main() {

    std::cout << "=== Task 5: Operator Precedence & Peak Logic ===" << std::endl;

    int hour = 9;

    bool isPeak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);

    std::cout << "Hour " << hour << " is peak time? " << (isPeak ? "YES" : "NO") << std::endl;



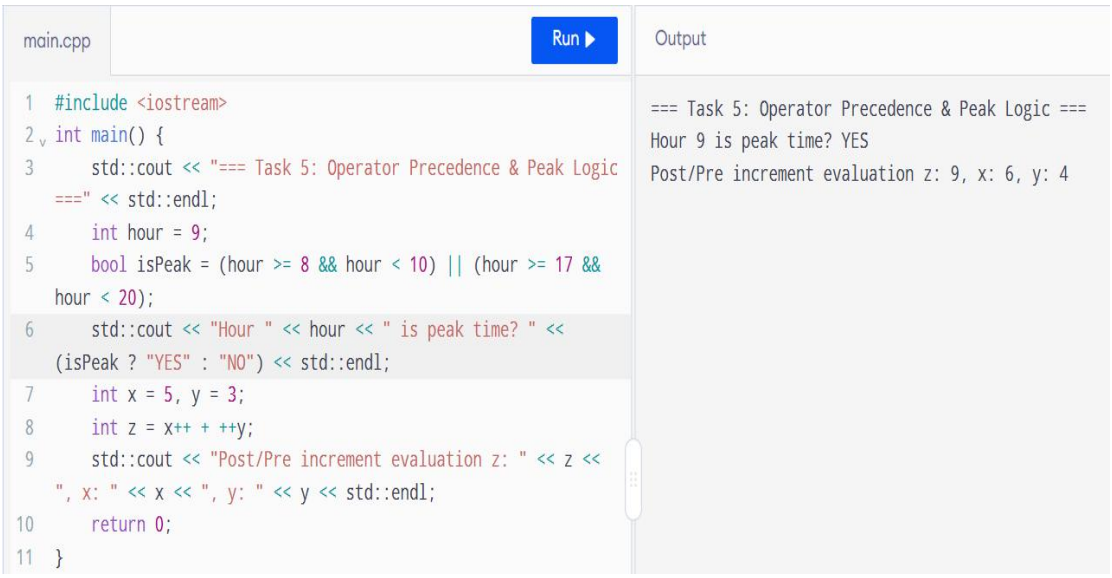
    int x = 5, y = 3;

    int z = x++ + ++y;

    std::cout << "Post/Pre increment evaluation z: " << z << ", x: " << x << ", y: " << y << std::endl;

    return 0;

}
```



Task 6: Control Structures & Fare Computation:-

PROGRAM CODE:-

```
#include <iomanip>
#include <iostream>


enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak) {
   long long base = 0;
   switch (z) {
      case Zone::Inner: base = BASE_PAISE[0]; break;
      case Zone::Middle: base = BASE_PAISE[1]; break;
      case Zone::Outer: base = BASE_PAISE[2]; break;
   }
   if (peak) {
      base += (base * PEAK_SURCHARGE_PERCENT) / 100;
   }
   return base;
}
bool canTravel(long long balance, long long f) {
   return (balance - f) >= MIN_BALANCE_PAISE;
```

```
}

int main() {

    std::cout << "=== Task 6: Switch & Control Structures ===" << std::endl;

    long long balance = 10000;

    struct Trip { Zone zone; bool peak; };

    Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle,
false}};

    for (const Trip& t : trips) {

        long long f = fare(t.zone, t.peak);

        if (canTravel(balance, f)) {

            balance -= f;

            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2) << std::setfill('0')
<< f % 100;

        } else {

            std::cout << "Denied (low balance)";

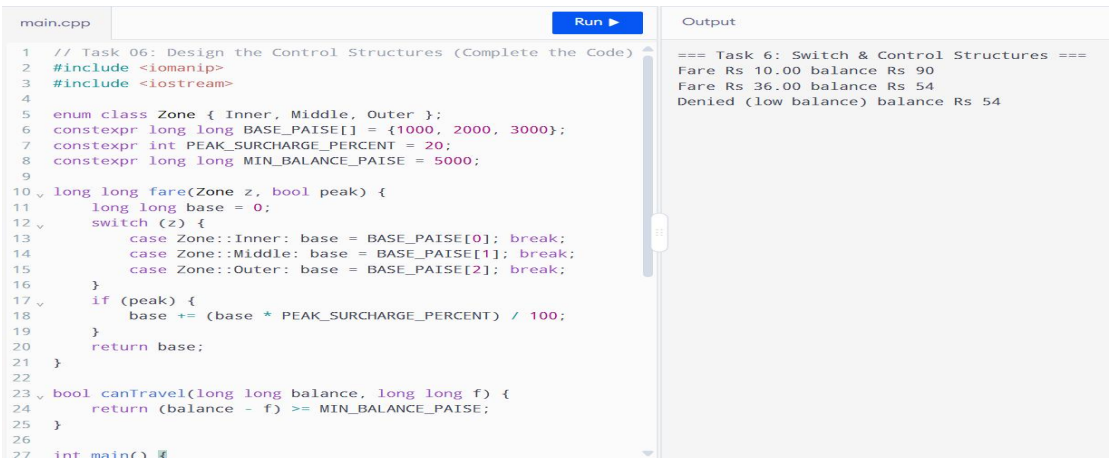
        }

        std::cout << " balance Rs " << balance / 100 << std::endl;

    }

    return 0;

}
```



Task 7: Procedural vs. Object-Oriented Style:-

PROGRAM CODE:-

```
#include <iostream>

class SmartCard {

private:

   long long balancePaise;

public:

   explicit SmartCard(long long initialBalance) : balancePaise(initialBalance) {}

   bool canTravel(long long fareAmount, long long minBalance) const {

      return (balancePaise - fareAmount) >= minBalance;

   }

   void deduct(long long amount) { balancePaise -= amount; }

   long long getBalance() const { return balancePaise; }

};

int main() {

   std::cout << "=== Task 7: Object-Oriented Encapsulation ===" << std::endl;

   SmartCard card(8000);

   long long fare = 2000;

   if (card.canTravel(fare, 5000)) {

      card.deduct(fare);

      std::cout << "Deduction successful. New balance: Rs " << card.getBalance()
/ 100 << std::endl;

   } else {

      std::cout << "Travel denied due to minimum balance constraint." <<
std::endl;

   }
```

return 0;

}

main.cpp

Run

```
    long long balancePaise;
public:
    explicit SmartCard(long long initialBalance) : balancePaise(initialBalance) {}
    bool canTravel(long long fareAmount, long long minBalance) const {
        return (balancePaise - fareAmount) >= minBalance;
    }
    void deduct(long long amount) { balancePaise -= amount; }
    long long getBalance() const { return balancePaise; }
};

int main() {
    std::cout << "=== Task 7: Object-Oriented Encapsulation ===" << std::endl;
    SmartCard card(8000);
    long long fare = 2000;
    if (card.canTravel(fare, 5000)) {
        card.deduct(fare);
        std::cout << "Deduction successful. New balance: Rs " << card.getBalance() / 100 << std::endl;
    } else {
        std::cout << "Travel denied due to minimum balance constraint." << std::endl;
    }
    return 0;
}
```

Output

```
=== Task 7: Object-Oriented Encapsulation ===
Deduction successful. New balance: Rs 60
```

Task 8: Safe Input Handling & Formatting:-

PROGRAM CODE:-

```
#include <iostream>

#include <limits>

#include <iomanip>

int main() {

   std::cout << "=== Task 8: Safe I/O & Formatting ===" << std::endl;

   int zoneChoice = 0;

   std::cout << "Select Zone (1: Inner, 2: Middle, 3: Outer): ";

   if (!(std::cin >> zoneChoice)) {

      std::cin.clear();

      std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

      std::cout << "Invalid input detected and cleared safely." << std::endl;

   } else {

      std::cout << "Zone code accepted: " << zoneChoice << std::endl;

   }

   return 0;

}
```



Task 9: Complete Implementation & Test Matrix:-

PROGRAM CODE:-

```
#include <iostream>

#include <cassert>

constexpr long long computeFare(int zone, bool peak) {

    long long base = (zone == 1) ? 1000 : (zone == 2) ? 2000 : 3000;

    if (peak) base += (base * 20) / 100;

    return base;

}

int main() {

    std::cout << "=== Task 9: Automated Test Matrix ===" << std::endl;

    assert(computeFare(1, false) == 1000);

    assert(computeFare(3, true) == 3600);

    std::cout << "All automated test cases passed successfully!" << std::endl;

    return 0;

}
```

main.cpp

Run ▶

```
1  // Task 09: Implement and Test the Fare Logic
2  #include <iostream>
3  #include <cassert>
4
5  constexpr long long computeFare(int zone, bool peak) {
6      long long base = (zone == 1) ? 1000 : (zone == 2) ? 2000 :
   3000;
7      if (peak) base += (base * 20) / 100;
8      return base;
9  }
10
11 int main() {
12     std::cout << "=== Task 9: Automated Test Matrix ===" <<
   std::endl;
13     assert(computeFare(1, false) == 1000);
14     assert(computeFare(3, true) == 3600);
15     std::cout << "All automated test cases passed
   successfully!" << std::endl;
16     return 0;
17 }
```

Output

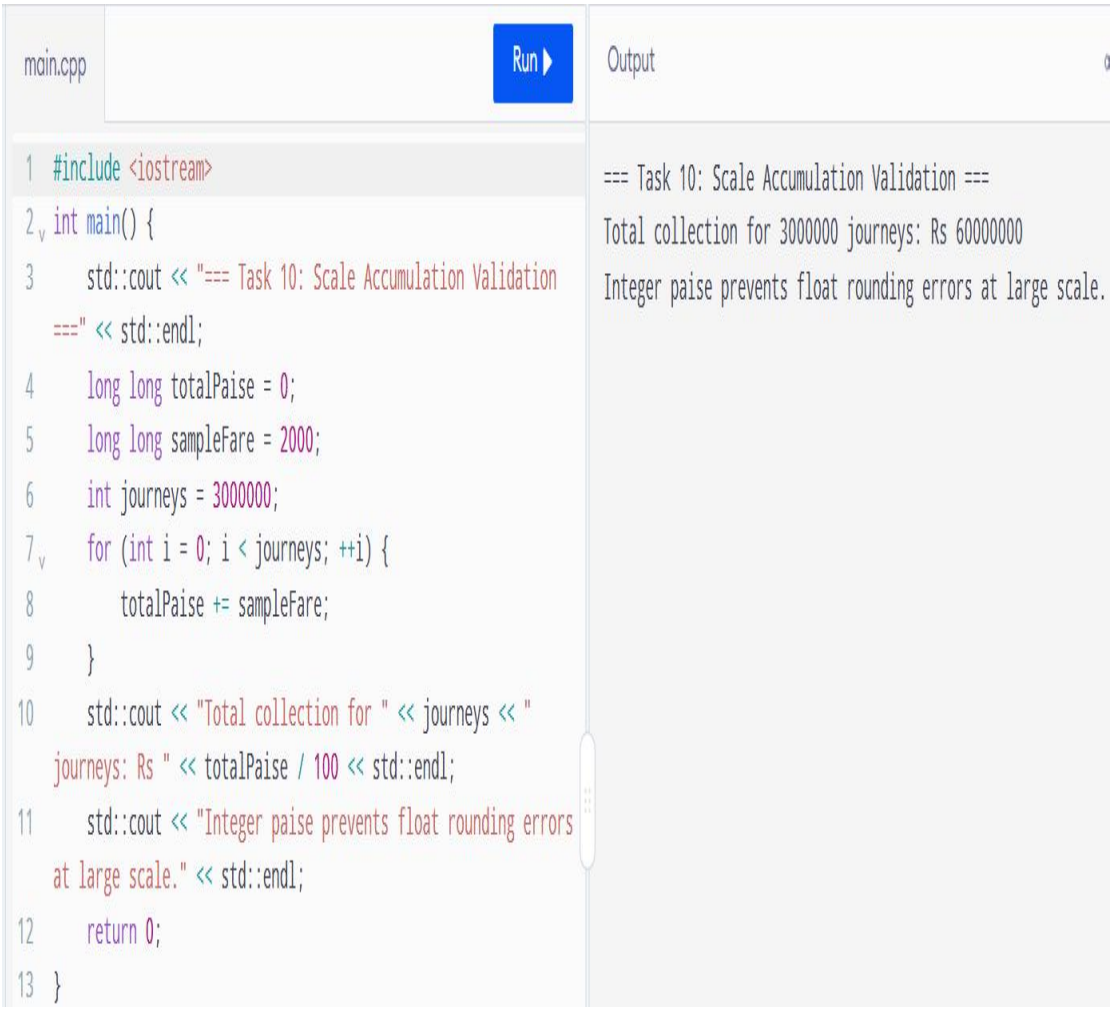
```
=== Task 9: Automated Test Matrix ===
All automated test cases passed successfully!
```

Task 10: Expectation vs. Execution Scale Check:-

PROGRAM CODE:-

```
#include <iostream>

int main() {
   std::cout << "=== Task 10: Scale Accumulation Validation ===" << std::endl;
   long long totalPaise = 0;
   long long sampleFare = 2000;
   int journeys = 3000000;
   for (int i = 0; i < journeys; ++i) {
      totalPaise += sampleFare;
   }
   std::cout << "Total collection for " << journeys << " journeys: Rs " <<
totalPaise / 100 << std::endl;
   std::cout << "Integer paise prevents float rounding errors at large scale." <<
std::endl;
   return 0;
}
```



Task 11: Final Design Recommendation & Architecture:-

PROGRAM CODE:-

```
// Task 11: Recommend and Defend the Final Design
#include <iostream>

int main() {
    std::cout << "=== Task 11: Final Architectural Defense ===" << std::endl;
    std::cout << "Recommendation: Combine constexpr fare tables, strong enum classes," << std::endl;
    std::cout << "encapsulated SmartCard state, and integer-paise accounting for zero defects." << std::endl;
    return 0;
}
```



Task 12: Day-4 Design Challenge (4th Zone, Discounts & Passes):-

PROGRAM CODE:-

```
#include <iostream>
enum class Zone { Inner = 1, Middle, Outer, Airport };
constexpr long long BASE_PAISE[] = { 1000, 2000, 3000, 6000 };
class Pass {
private:
   Zone z1, z2;
public:
   Pass(Zone a, Zone b) : z1(a), z2(b) {}
   bool covers(Zone z) const { return z == z1 || z == z2; }
};
long long getFare(Zone z, bool peak, bool weekend, const Pass* pass) {
   if (pass && pass->covers(z)) return 0; // Covered by monthly pass
   long long base = BASE_PAISE[static_cast<int>(z) - 1];
   if (weekend) {
      base -= (base * 10) / 100; // 10% Weekend discount
   } else if (peak) {
      base += (base * 20) / 100; // 20% Peak surcharge
   }
   return base;
}
int main() {
   std::cout << "=== Task 12: Day-4 Design Challenge ===" << std::endl;
   Pass myPass(Zone::Inner, Zone::Middle);
   std::cout << "Airport Zone Fare (Weekend): Rs " << getFare(Zone::Airport,
false, true, nullptr) / 100.0 << std::endl;
   std::cout << "Inner Zone Fare with Pass: Rs " << getFare(Zone::Inner, true,
false, &myPass) / 100.0 << std::endl;
   return 0;
}
```

